

Secure Peer-to-Peer File Sharing System

Advanced Operating Systems — Assignment 3

Systems & Network Security Integration

Roll Number: 2025201067

IIIT Hyderabad

Monsoon 2025

Contents

1	Abstract	3
2	System Architecture	3
2.1	High-Level Overview	3
2.2	Architecture Diagram	3
2.3	Communication Channels	4
3	Security Design	4
3.1	Threat Model	4
3.2	Mitigation Summary	4
3.3	TLS 1.2+ Encryption	5
3.4	PKI and Certificate Management	5
3.5	Mutual TLS (mTLS) for P2P	6
3.6	RSA-2048 Digital Signatures	6
3.7	Salted Password Hashing	7
3.8	SHA-256 File Integrity	7
4	Network Protocol Design	7
4.1	Client $\leftrightarrow$ Tracker Protocol (over TLS)	7
4.2	Peer-to-Peer Protocol (over mTLS)	7
5	Key Algorithms	8
5.1	Rarest Piece First	8
5.2	Multi-Threaded Download Engine	8
6	Data Structures	8
6.1	Tracker Side	8
6.2	Client Side	9
7	Tracker Synchronization	9
8	Security Analysis	10
8.1	Properties Achieved	10
8.2	Remaining Limitations	10

9	Testing Procedures	10
9.1	Security Verification	10
9.2	End-to-End Test	10
10	Conclusion	11

1 Abstract

This report presents the design and implementation of a **secure peer-to-peer (P2P) file sharing system** inspired by BitTorrent. Unlike traditional P2P systems that transmit data in plaintext, our system integrates a comprehensive security layer consisting of:

- **TLS 1.2+** encryption on all communication channels
- **Mutual TLS (mTLS)** with X.509 certificates for peer authentication
- **RSA-2048 digital signatures** on file metadata to prevent tampering
- **SHA-256** cryptographic hashing for file integrity verification
- **Salted password hashing** to protect stored credentials
- A full **PKI (Public Key Infrastructure)** with the tracker acting as a Certificate Authority

The system supports multi-seeder downloads with a rarest-piece-first strategy, partial file serving, and two-tracker redundancy with automatic state synchronization.

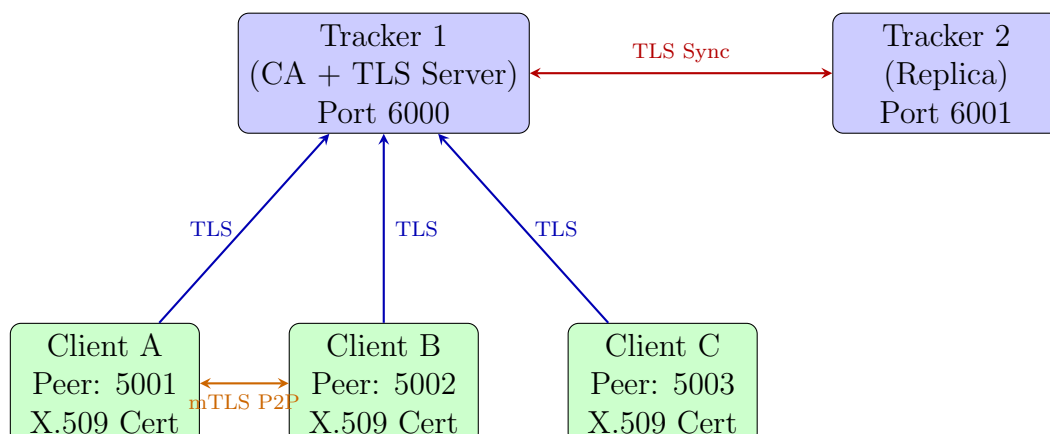
2 System Architecture

2.1 High-Level Overview

The system consists of three types of entities:

1. **Tracker (Server):** Centralized coordination server managing user accounts, groups, file metadata, seeder information, and acting as a Certificate Authority (CA).
2. **Client:** Peer application that uploads, downloads, and shares files. Each client runs an embedded peer server.
3. **Certificate Authority:** Embedded within the tracker; issues X.509 certificates to authenticated users.

2.2 Architecture Diagram



2.3 Communication Channels

Channel	Security	Purpose
Client $\leftrightarrow$ Tracker	TLS 1.2+ (server auth)	Commands, metadata, cert issuance
Client $\leftrightarrow$ Client	mTLS (mutual auth)	Piece transfer, bitmap queries
Tracker $\leftrightarrow$ Tracker	TLS 1.2+	State synchronization

Table 1: Communication channels and their security properties

3 Security Design

3.1 Threat Model

We consider the following threats:

1. **Eavesdropping:** An attacker on the network can observe all traffic between any two entities.
2. **Man-in-the-Middle (MitM):** An attacker can intercept and modify messages in transit.
3. **Impersonation:** A malicious node can pretend to be a legitimate peer to serve corrupted file pieces.
4. **Metadata Tampering:** A compromised tracker could alter piece hashes to redirect downloads to malware.
5. **Credential Theft:** An attacker with access to the tracker’s memory could steal user passwords.

3.2 Mitigation Summary

Threat	Mitigation	Mechanism
Eavesdropping	Channel encryption	TLS 1.2+ (AES-256-GCM)
MitM	Server authentication	X.509 certificates, CA verification
Impersonation	Peer authentication	Mutual TLS with tracker-issued certs
Metadata Tampering	Digital signatures	RSA-2048 with SHA-256 digest
Credential Theft	Password hashing	SHA-256(salt password)

Table 2: Threat mitigations implemented in the system

3.3 TLS 1.2+ Encryption

All TCP connections in the system are wrapped with TLS using OpenSSL. The implementation uses:

- `TLS_server_method()` and `TLS_client_method()` for version-agnostic TLS
- Minimum protocol version set to TLS 1.2 via `SSL_CTX_set_min_proto_version()`
- Server certificate verification via `SSL_CTX_load_verify_locations()` with the CA cert

Listing 1: TLS Context Creation (Server Side)

```

1 SSL_CTX* create_server_tls_ctx(const char* cert,
2                               const char* key,
3                               const char* ca) {
4     SSL_CTX* ctx = SSL_CTX_new(TLS_server_method());
5     SSL_CTX_set_min_proto_version(ctx, TLS1_2_VERSION);
6     SSL_CTX_use_certificate_file(ctx, cert, SSL_FILETYPE_PEM);
7     SSL_CTX_use_PrivateKey_file(ctx, key, SSL_FILETYPE_PEM);
8     SSL_CTX_load_verify_locations(ctx, ca, nullptr);
9     SSL_CTX_set_verify(ctx, SSL_VERIFY_PEER, nullptr);
10    return ctx;
11 }
```

3.4 PKI and Certificate Management

The system implements a complete PKI chain:

1. **Certificate Authority (CA):** A self-signed RSA-2048 root certificate generated by `generate_certs.sh`. The CA's private key is loaded by the tracker at startup.
2. **Tracker Certificate:** An X.509 certificate signed by the CA, presented to clients during TLS handshake. Clients verify this certificate against the CA cert.
3. **Client Certificates (On-the-fly):** When a user successfully logs in, the tracker:
 - (a) Generates a new RSA-2048 key pair for the client
 - (b) Creates an X.509 certificate with CN=<username>
 - (c) Signs it with the CA's private key using SHA-256
 - (d) Sends the PEM-encoded certificate and private key to the client over the TLS channel

Listing 2: On-the-fly Certificate Generation (Tracker)

```

1 // Generate RSA-2048 key pair for client
2 EVP_PKEY* client_key = EVP_PKEY_new();
3 RSA* rsa = RSA_generate_key(2048, RSA_F4, NULL, NULL);
4 EVP_PKEY_assign_RSA(client_key, rsa);
5
```

```

6 // Create X509 certificate
7 X509* cert = X509_new();
8 X509_set_version(cert, 2); // v3
9 X509_NAME* name = X509_get_subject_name(cert);
10 X509_NAME_add_entry_by_txt(name, "CN", MBSTRING_ASC,
11     (unsigned char*)username.c_str(), -1, -1, 0);
12
13 // Sign with CA key (SHA-256 digest)
14 X509_sign(cert, ca_key, EVP_sha256());

```

3.5 Mutual TLS (mTLS) for P2P

When Client A wants to download a piece from Client B:

1. Client A connects to Client B's peer server via TCP
2. Both sides perform a TLS handshake presenting their X.509 certificates
3. Each side verifies the peer's certificate is signed by the trusted CA
4. If verification fails, the connection is rejected
5. Piece data is transferred over the encrypted, authenticated channel

This prevents unauthorized peers from joining the swarm and serving malicious pieces.

3.6 RSA-2048 Digital Signatures

File metadata integrity is protected by RSA-2048 digital signatures:

1. **Upload:** The client constructs a metadata blob:
filename|filesize|file_hash|piece_hash_1|...|piece_hash_N
2. **Signing:** The client computes `RSA_Sign(SHA256(metadata_blob), private_key)`
3. **Storage:** The hex-encoded signature and the client's public key are sent to the tracker along with the file metadata.
4. **Verification:** When another client requests a download, the tracker includes the signature and public key in the response. The downloader verifies:
`RSA_Verify(SHA256(metadata_blob), signature, public_key)`

Listing 3: RSA-2048 Digital Signature

```

1 string rsa_sign(const string& priv_pem, const string& data) {
2     // Load private key from PEM
3     BIO* bio = BIO_new_mem_buf(priv_pem.c_str(), -1);
4     EVP_PKEY* pkey = PEM_read_bio_PrivateKey(bio, ...);
5
6     // Sign with SHA-256 digest
7     EVP_MD_CTX* ctx = EVP_MD_CTX_new();
8     EVP_SignInit(ctx, EVP_sha256());

```

```
9     EVP_SignUpdate(ctx, data.c_str(), data.size());
10     EVP_SignFinal(ctx, sig_buf, &sig_len, pkey);
11
12     return hex_encode(sig_buf, sig_len);
13 }
```

3.7 Salted Password Hashing

Passwords are never stored in plaintext on the tracker:

1. **Registration:** A random 16-byte salt is generated using `RAND_bytes()`. The password is hashed as `SHA256(salt || password)` and stored alongside the salt.
2. **Login:** The provided password is hashed with the stored salt and compared against the stored hash.
3. **Sync:** The hashed password and salt (not the plaintext) are synchronized to the peer tracker.

This means even if an attacker gains access to the tracker's memory, they cannot recover plaintext passwords.

3.8 SHA-256 File Integrity

All file integrity verification uses SHA-256 (replacing the cryptographically broken SHA-1):

- **Piece-level:** Each 512KB chunk is independently hashed with SHA-256 during upload. During download, each received piece is re-hashed and compared against the expected hash.
- **Full-file:** A streaming SHA-256 hash of the entire file is computed during upload using `SHA256_Init/Update/Final`, processing chunks sequentially without loading the full file into memory.
- **Hash length:** 64 hexadecimal characters (256 bits) vs. SHA-1's 40 characters (160 bits).

4 Network Protocol Design

4.1 Client ↔ Tracker Protocol (over TLS)

All messages are newline-terminated and transmitted over TLS-encrypted channels.

4.2 Peer-to-Peer Protocol (over mTLS)

Each piece request uses a short-lived mTLS connection:

- `GET_PIECE <filename> <index> → PIECE_LEN <bytes> + raw data`
- `GET_BITMAP <filename> <total> → BITMAP <bitstring>`

Request	Response
<code>create_user <uid> <pwd></code>	User created successfully
<code>login <uid> <pwd></code>	Login successful CERT <hex> KEY <hex>
<code>UPLOAD_FILE <gid> <fname></code>	SUCCESS: File metadata uploaded
<code><size> <hash> <pieces></code>	
<code><sig> <pubkey></code>	
<code>DOWNLOAD_FILE <gid></code>	DOWNLOAD_INFO <size> <seeders>
<code><fname></code>	<hashes> SIG <sig> PUBKEY <pubkey>

Table 3: Key protocol messages (security-relevant fields highlighted)

5 Key Algorithms

5.1 Rarest Piece First

Before downloading, the client queries all seeders for their piece bitmaps:

1. For each seeder, send `GET_BITMAP` and receive a binary string (e.g., "110101")
2. Compute rarity: for each piece, count how many seeders have it
3. Worker threads select the piece with the **lowest rarity** (fewest sources)
4. A random seeder is chosen from those that have the selected piece

This strategy ensures rare pieces are replicated first, preventing the “last piece” problem.

5.2 Multi-Threaded Download Engine

- Thread count = `min(hardware_concurrency, total_pieces)`
- Each worker independently claims a piece (under mutex), downloads it via mTLS, verifies its SHA-256 hash, and writes to disk using `pwrite()` at the correct byte offset
- Failed pieces are retried up to 5 times before aborting
- A `condition_variable` coordinates completion between workers and the manager thread

6 Data Structures

6.1 Tracker Side

Listing 4: Core Data Structures (Tracker)

```

1 struct User {
2     string username;
3     string password;           // SHA-256 hash (never plaintext)

```

```

4     string password_salt; // 16-byte random salt (hex)
5     bool logged_in;
6     string ip; int port;
7 };
8
9 struct FileMetadata {
10     size_t filesize;
11     string file_hash;           // SHA-256 of full file
12     vector<string> piece_hashes; // SHA-256 per 512KB chunk
13     vector<string> seeders;
14     string signature;           // RSA-2048 digital signature
15     string uploader_pubkey;     // Uploader's RSA public key
16 };
17
18 struct Group {
19     string group_id, owner;
20     vector<string> members, pending_requests;
21     map<string, FileMetadata> files;
22 };

```

6.2 Client Side

Listing 5: Download State (Client)

```

1 struct DownloadState {
2     vector<PieceStatus> piece_status; // NEEDED/IN_FLIGHT/HAVE
3     atomic<int> pieces_completed;
4     vector<string> seeder_addresses;
5     vector<string> piece_hashes;       // Expected SHA-256
6     vector<int> piece_rarity;          // Seeder count per piece
7     vector<vector<int>> piece_seeder_map;
8     bool completed, failed;
9 };

```

7 Tracker Synchronization

The two-tracker system uses bidirectional TLS-encrypted persistent connections:

1. **Listener Thread:** Listens on `sync_port`, accepts TLS connections
2. **Connector Thread:** Actively connects to peer's `sync_port` with 3-second retry backoff
3. **Heartbeat:** If no data for 5 seconds, sends HEARTBEAT to detect dead connections
4. **ACK:** Each sync message is acknowledged with `ACK:<original_message>`

Sync messages are pipe-delimited (e.g., `SYNC|CREATE_USER|uid|hash|salt`).

8 Security Analysis

8.1 Properties Achieved

1. **Confidentiality:** All data in transit is encrypted with TLS (AES-256-GCM cipher suites). Passwords are never transmitted or stored in plaintext.
2. **Integrity:** File pieces are verified with SHA-256 hashes. File metadata is signed with RSA-2048 to prevent tampering.
3. **Authentication:** The tracker authenticates to clients via its X.509 certificate. Peers authenticate to each other via mTLS with tracker-issued certificates.
4. **Non-repudiation:** RSA digital signatures on file metadata ensure the uploader cannot deny uploading a file with specific contents.

8.2 Remaining Limitations

1. **Single CA:** The tracker is the sole CA. If compromised, all trust is broken.
2. **No Certificate Revocation:** There is no CRL or OCSP mechanism to revoke compromised client certificates.
3. **In-memory state:** Tracker state (including the CA private key) is in-memory. Restarting loses all data.
4. **No forward secrecy guarantee:** While TLS 1.2+ supports forward secrecy cipher suites, we do not enforce them.

9 Testing Procedures

9.1 Security Verification

Listing 6: Verifying TLS is Active

```
1 # Use openssl s_client to verify TLS handshake
2 openssl s_client -connect 127.0.0.1:6000 \
3     -CAfile certs/ca.crt
4
5 # Verify with tcpdump that traffic is encrypted
6 sudo tcpdump -i lo0 -X port 6000
7 # Should show encrypted (non-readable) payload
```

9.2 End-to-End Test

Listing 7: Complete Test Flow

```
1 # Generate certificates
2 make certs
3
4 # Build
```

```
5 make
6
7 # Start trackers (2 terminals)
8 ./server/tracker tracker_info.txt 1
9 ./server/tracker tracker_info.txt 2
10
11 # Start clients (2 terminals)
12 ./client/client 127.0.0.1:5001 tracker_info.txt
13 > create_user alice pass123
14 > login alice pass123
15 > create_group g1
16 > upload_file g1 /path/to/testfile.pdf
17
18 ./client/client 127.0.0.1:5002 tracker_info.txt
19 > create_user bob pass456
20 > login bob pass456
21 > join_group g1
22 # (Accept from alice's terminal)
23 > download_file g1 testfile.pdf /tmp/downloaded.pdf
24 > show_downloads
```

10 Conclusion

This project demonstrates the integration of multiple cryptographic techniques into a practical distributed system:

- **Transport Layer Security (TLS)** provides confidentiality and server authentication
- **Mutual TLS (mTLS)** extends authentication to peer-to-peer connections
- **RSA-2048 Digital Signatures** ensure file metadata integrity and non-repudiation
- **SHA-256** replaces the broken SHA-1 for collision-resistant file integrity
- **Salted Password Hashing** protects user credentials at rest
- **PKI with on-the-fly certificate issuance** provides a scalable trust model

The result is a P2P file sharing system where all communication is encrypted, all peers are authenticated, all file metadata is cryptographically signed, and all credentials are securely stored — representing a significant improvement over traditional plaintext P2P protocols.